Gitの中身

慶應義塾大学理工学部物理情報工学科
渡辺

今回の目標

Gitの内部実装について知る

- .gitディレクトリの中身
- Gitのオブジェクト
- Gitの参照
- インデックス

はじめに

今日の話を覚える必要はありません

内部実装を知らなくてもGitは全く問題なく使える
将来、内部実装が変更される可能性もある

はじめに

覚える必要がないなら、なんで内部実装なんか講義で紹介するの？

「全てのソフトウェアには実装がある」という感覚を持ってもらうためです。

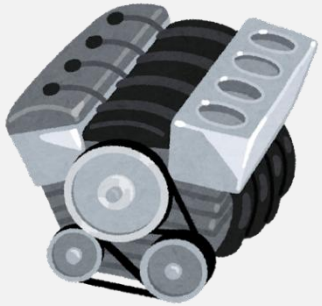
目的と実装

目的：食品を加熱する

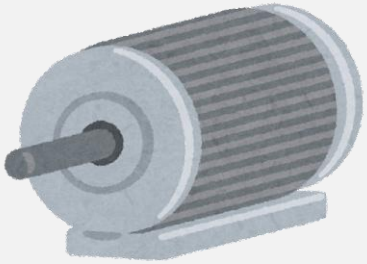
電子レンジ

オーブン

目的：自動車を駆動する



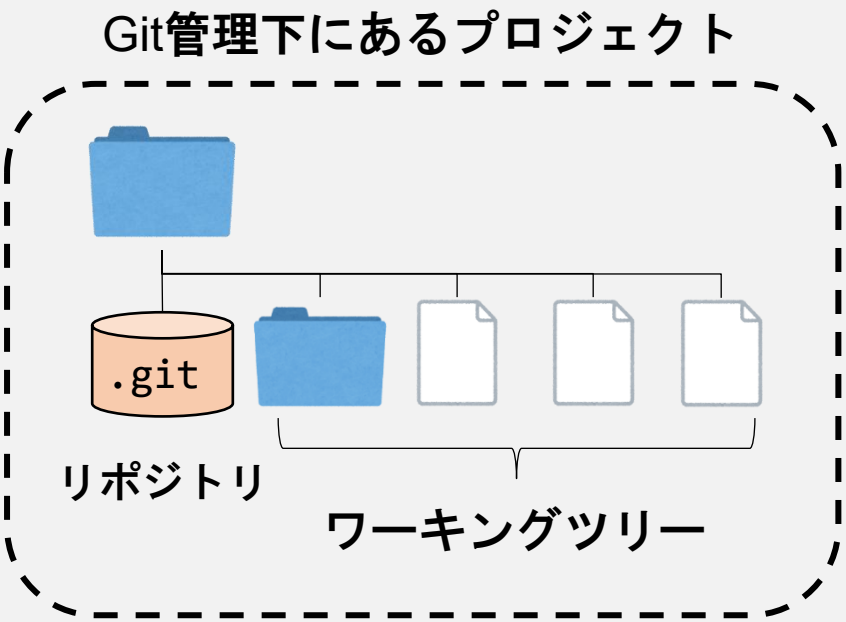
エンジン



モーター

同じ目的でも、異なる実装方法があり得る

.gitの中身



Gitが管理する情報は全て
.gitディレクトリに入っている

適当なリポジトリで.gitの中身を表示してみる

```
$ ls .git
COMMIT_EDITMSG  HEAD        branches/  description  index  logs/     packed-refs
FETCH_HEAD      ORIG_HEAD   config     hooks/       info/  objects/  refs/
```

.gitの中身

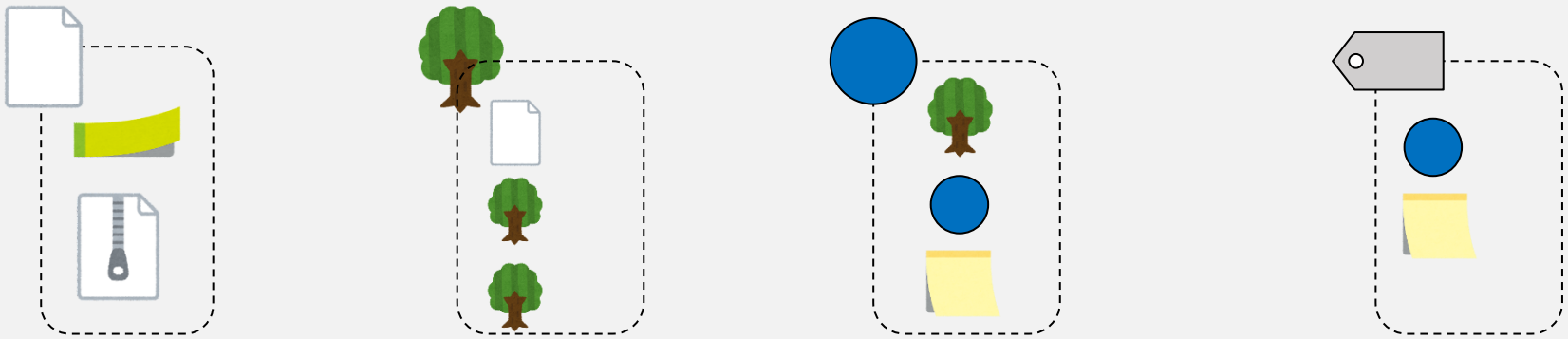
.gitの中身のうち、本講義では以下について説明する

- HEAD：カレントブランチ(HEAD)の情報を保存するファイル
- index：インデックスの情報を保存するファイル
- config：リモートや上流ブランチ等の情報を保存するファイル
- refs：ブランチの情報を保存するディレクトリ
- objects：コミットなどのオブジェクトを保存するディレクトリ

Gitのオブジェクト

.git/objectsにはGitが管理するオブジェクトが格納されている
Gitのオブジェクトは以下の四種類

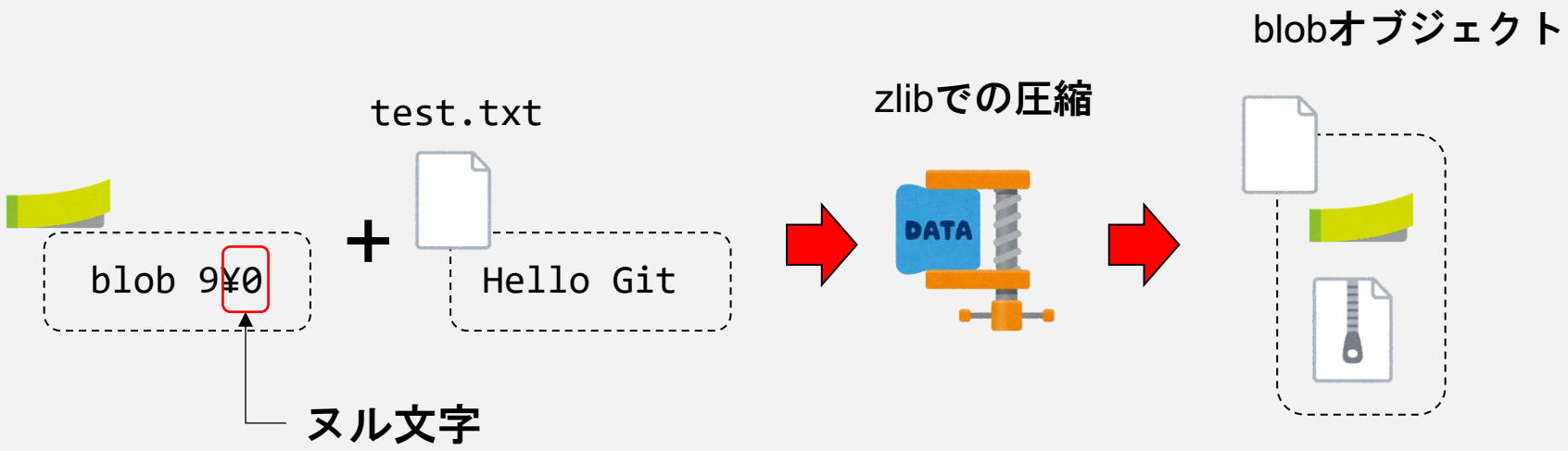
blobオブジェクト　treeオブジェクト　コミットオブジェクト　タグオブジェクト



- blobオブジェクト：ファイルを圧縮したもの。「ファイル」に相当
- treeオブジェクト：blobやtreeオブジェクトを管理する。「ディレクトリ」に相当
- コミットオブジェクト：treeオブジェクトを包んだもの。コミットのスナップショットに対応するtreeオブジェクトに、親コミットやコミットメッセージなどを付加する。
- タグオブジェクト：他のGitオブジェクト(多くはコミットオブジェクト)を包んでタグ付与者やタグメッセージをつけたもの。

blobオブジェクト

Blob (Binary Large Object)は、ファイルを保存するためのオブジェクト
「blob」というテキストと、ファイルサイズをヘッダとして付与して
ファイルをzlibで圧縮したもの



blobオブジェクト

blobオブジェクトは、初めてファイルをインデックスにステージングした時に作成される

「Hello Git」という中身を持つファイルをステージングしてみる

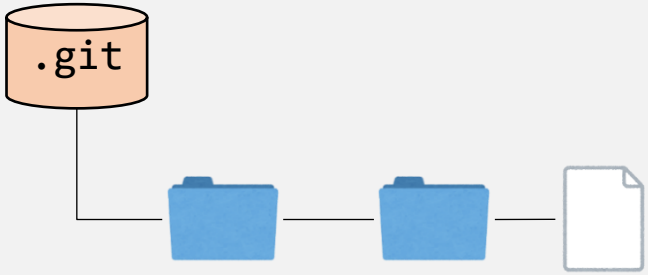
```
echo -n "Hello Git" > test.txt
git add test.txt
```

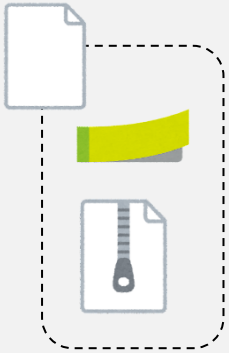
以下のハッシュ(後述)を持つblobオブジェクト作られる

e51ca0d0b8c5b6e02473228bbf876ba000932e96

blobオブジェクト

blobオブジェクトは、.git/objects以下に、「頭二文字」のディレクトリの下に、残りの38文字のファイルとして保存される





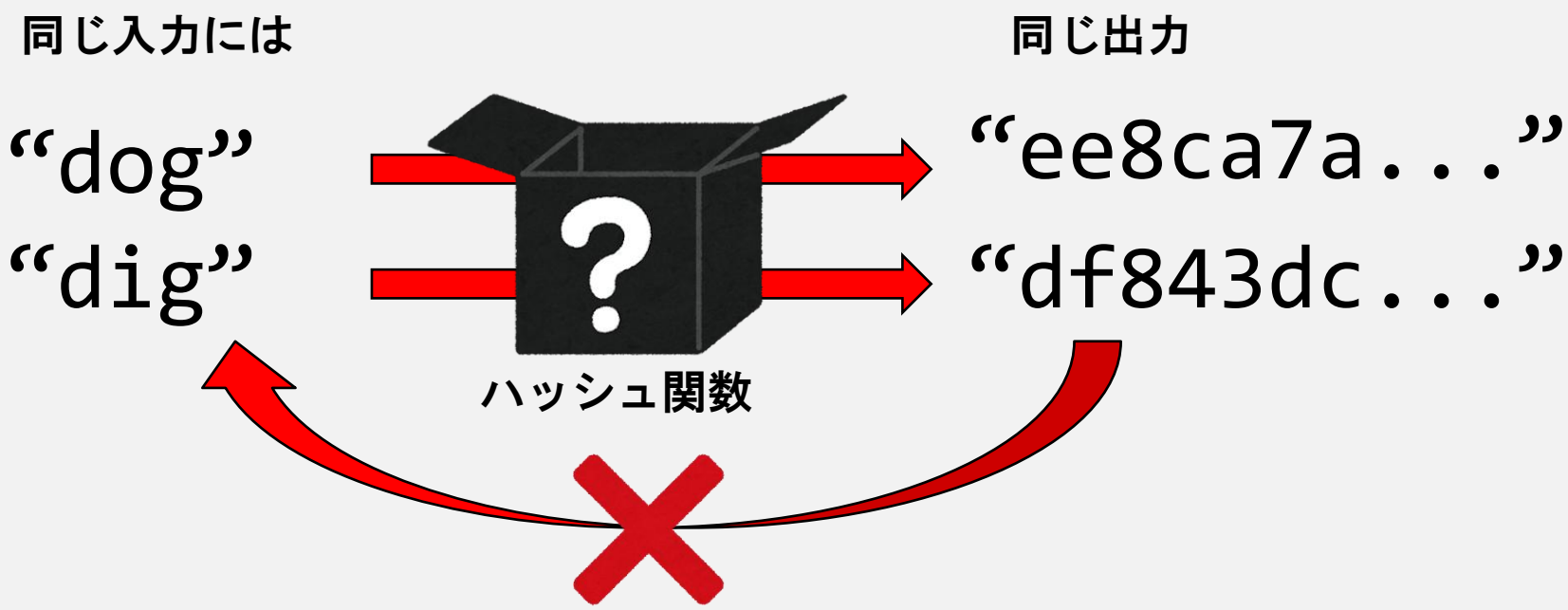
e51ca0d0b8c5b6e02473228bbf876ba000932e96

Gitのblobオブジェクト名やコミットハッシュに現れる40桁の文字列はSHA-1ハッシュ値

SHA-1 ハッシュ

ハッシュ値とは以下の性質を満たすもの

- 任意の長さのデータから特定の計算手順で求められる固定長の値
- 同じ入力には同じ出力を返す
- 少しでも入力が変化すると、ハッシュ値が大きく変化する
- 入力データからハッシュ値を得るのは容易だが、あるハッシュ値を出力とするような入力を探すのは極めて困難(**強衝突耐性**)



ハッシュ値から対応する入力を推定するのは困難

SHA-1 ハッシュ

- Secure Hash Algorithm (SHA)
- SHA-1はSHAシリーズの一つ
- 任意の入力に160ビットの出力を返す
- 16進数で表すと40桁
- SHA-1の強衝突耐性は突破されている
  (セキュリティ用途には向かない)

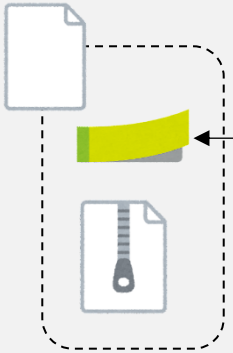
shasumというコマンドでSHA-1ハッシュを求めることができる

```
$ shasum test.txt
06f7894ea013cdaac36c5878694ef99a32b1e11d *test.txt
```

blobオブジェクトを作ってみる

ファイル名の取得

```
$ { echo -en 'blob 9¥0';cat test.txt;} | shasum
e51ca0d0b8c5b6e02473228bbf876ba000932e96 *-
```



この部分はヘッダ(blob+ファイルサイズ)

ヘッダをつけてSHA-1値を計算するコマンド

```
$ git hash-object test.txt
e51ca0d0b8c5b6e02473228bbf876ba000932e96
```

blobオブジェクトを作ってみる

blobオブジェクトを作るPythonスクリプト

```
import zlib
content = "Hello Git" # ファイルの中身

# ヘッダ付与
store = f"blob {len(content)}¥0{content}".encode("utf-8")

data = zlib.compress(store, level=1) # 圧縮
print(bytes.hex(data))       # 中身の表示(16進ダンプ)
```

実行結果

```
$ python3 test.py
78014bcac94f52b064f048cdc9c95770cf2c01002b750531
```

実際に作られたblobオブジェクトの中身の16進ダンプ

```
$ od -tx1 .git/objects/e5/1ca0d0b8c5b6e02473228bbf876ba000932e96
0000000 78 01 4b ca c9 4f 52 b0 64 f0 48 cd c9 c9 57 70
0000020 cf 2c 01 00 2b 75 05 31
```

コミットオブジェクト

先ほどgit addだけしていたものをコミットしてみる

```
$ git commit -m "initial commit"
[main (root-commit) ca70291] initial commit
 1 file changed, 1 insertion(+)
 create mode 100644 test.txt
```

新たにコミットオブジェクトca70291が作られた
この状態で.git/objectには3つのオブジェクトが保存されている

コミットオブジェクト

```
git cat-file -t オブジェクト名
```

オブジェクトのタイプを表示する

```
$ git cat-file -t ca70291
commit
$ git cat-file -t dd1d7ee
tree
$ git cat-file -t e51ca0d
blob
```

コミットオブジェクト

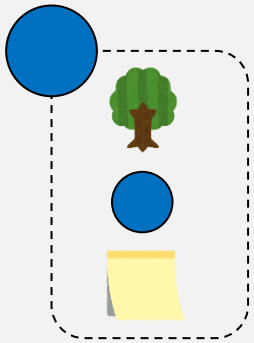
```
git cat-file -p オブジェクト名
```

オブジェクトの中身を表示する

```
$ git cat-file -p ca70291
tree dd1d7ee1e23a241a3597a0d0be5139a997fc29c8
author H. Watanabe <kaityo256@example.com> 1632060650 +0900
committer H. Watanabe <kaityo256@example.com> 1632060650 +0900

initial commit
```

コミットオブジェクト



スナップショットを表すtreeオブジェクト
親コミット(root-commitなら無し)
コミットした人の情報
コミットメッセージ

コミットオブジェクト

何か修正してコミットする

```
$ echo "Hello commit object" >> test.txt
$ git commit -am "update"
[main 1f620eb] update
 1 file changed, 1 insertion(+), 1 deletion(-)
```

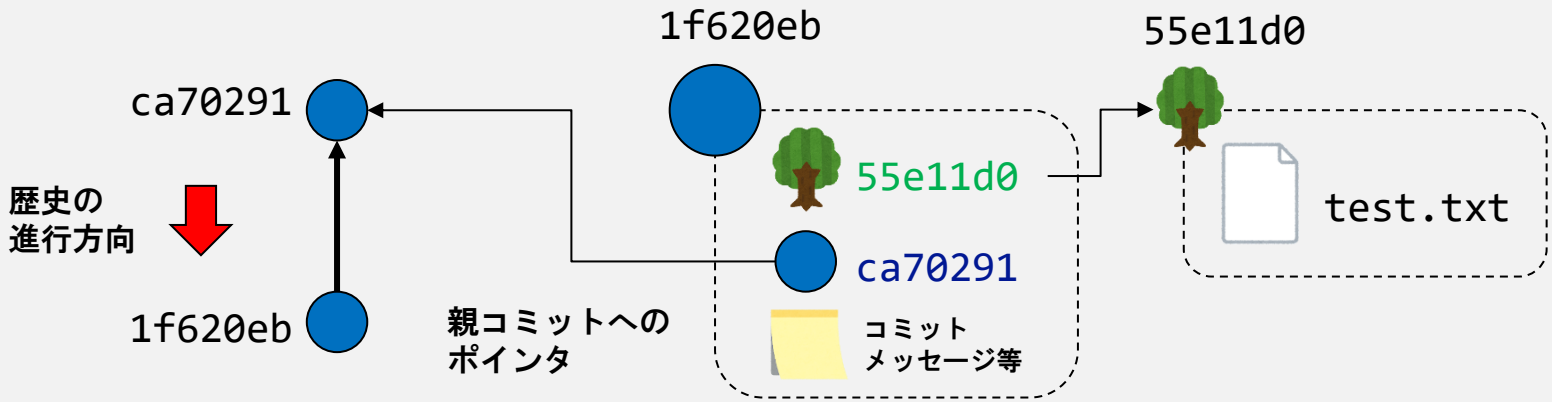
新たにコミットオブジェクト「1f620eb」ができたので、中身を見る

```
$ git cat-file -p 1f620eb
tree 55e11d02569af14b5d29fe56fd44c1cc32c55e72
parent ca70291031230dde40264d62b6e8d2424e2c9366
author H. Watanabe <kaityo256@example.com> 1630738892  +0900
committer H. Watanabe <kaityo256@example.com> 1630738892 +0900

update
```

スナップショットを表すtreeオブジェクトは「55e11d0」
親コミットは「ca7029」

コミットオブジェクト



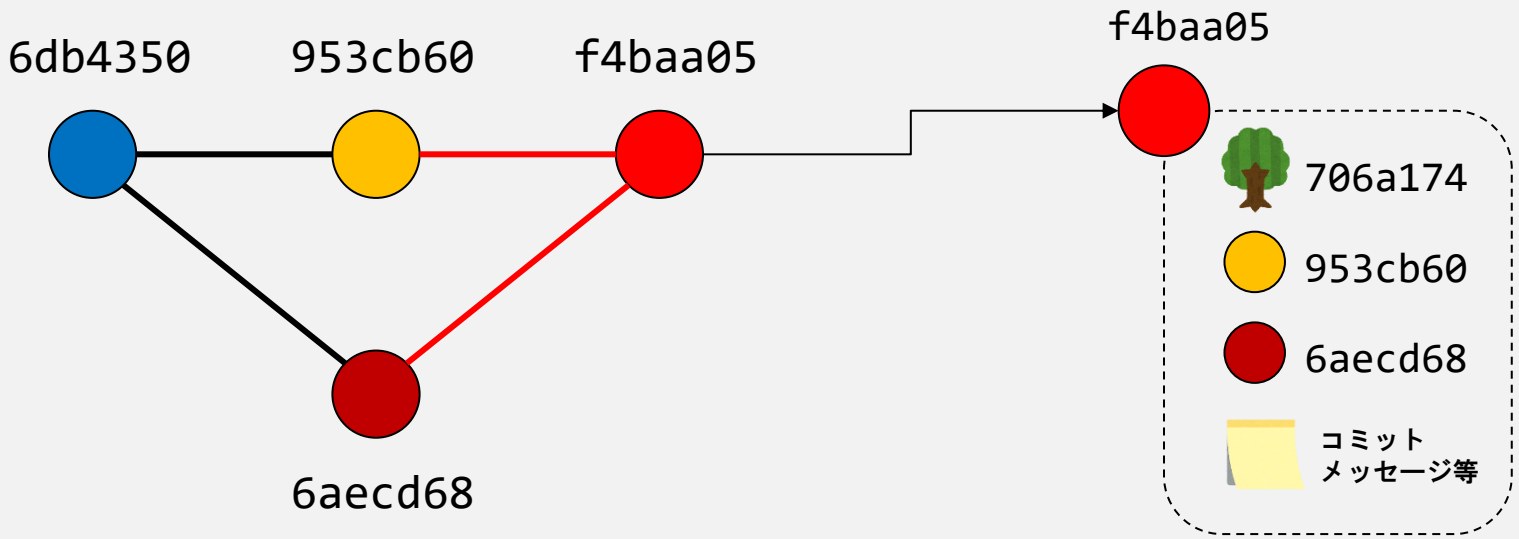

```
$ git cat-file -p 1f620eb
tree 55e11d02569af14b5d29fe56fd44c1cc32c55e72
parent ca70291031230dde40264d62b6e8d2424e2c9366
author H. Watanabe <kaityo256@example.com> 1630738892  +0900
committer H. Watanabe <kaityo256@example.com> 1630738892 +0900

update
```

スナップショットを表すtreeオブジェクトは「55e11d0」
親コミットは「ca7029」

マージコミット

マージコミットは親コミットを二つ持つ

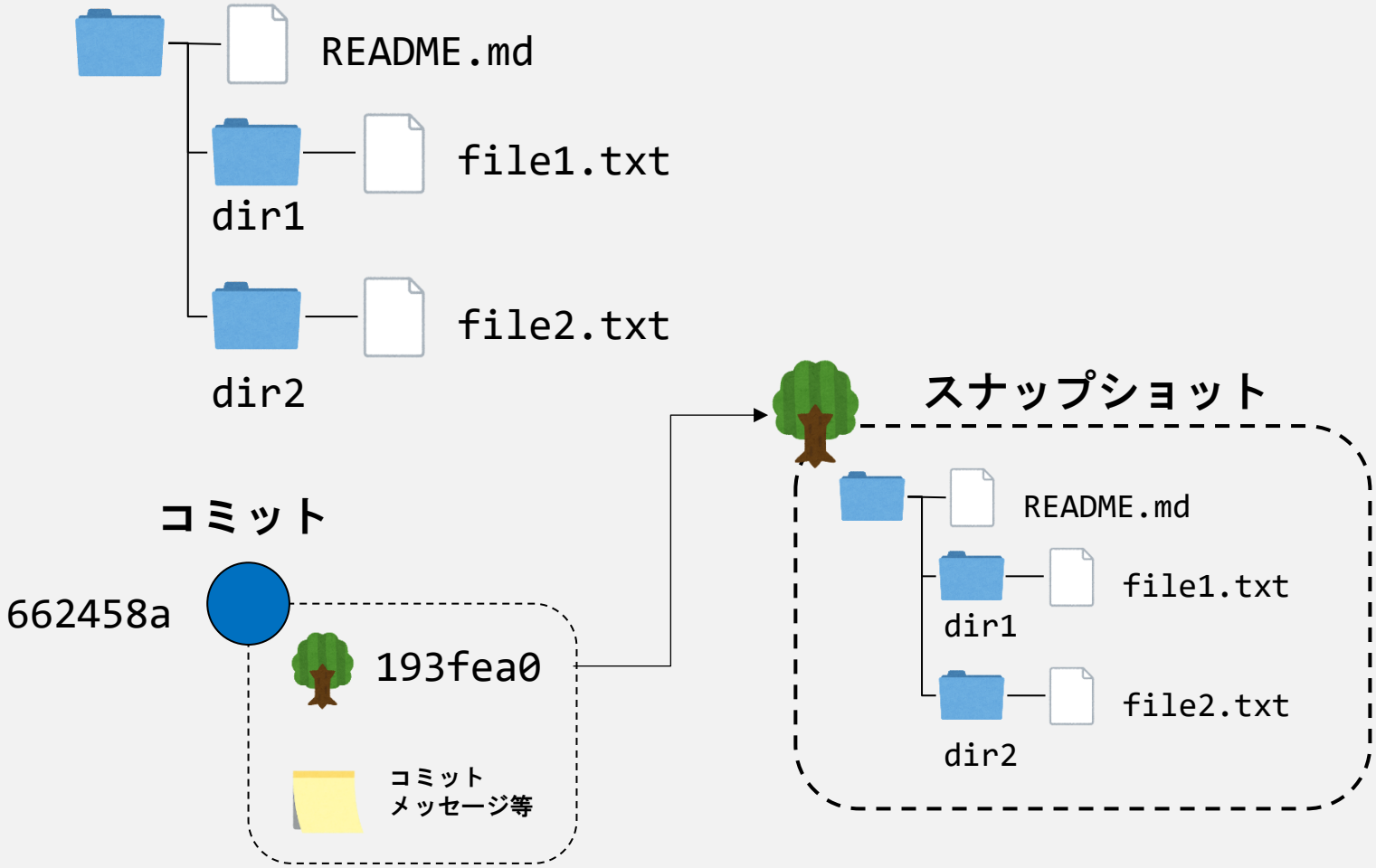



```
$ git cat-file -p f4baa05
tree 706a1741c1d94977ba496449d80ab848ca945e14
parent 953cb6056e5f0437f0d4e102f232d8eb705f6428
parent 6aecd68aa423651edda9d22e20925314ff3e8386
author H. Watanabe <kaityo256@example.com> 1630743012 +0900
committer H. Watanabe <kaityo256@example.com> 1630743012 +0900

Merge branch 'branch'
```

treeオブジェクト

以下のようなディレクトリ構造を持つプロジェクトがあり、
対応するコミットハッシュが662458aであるとする

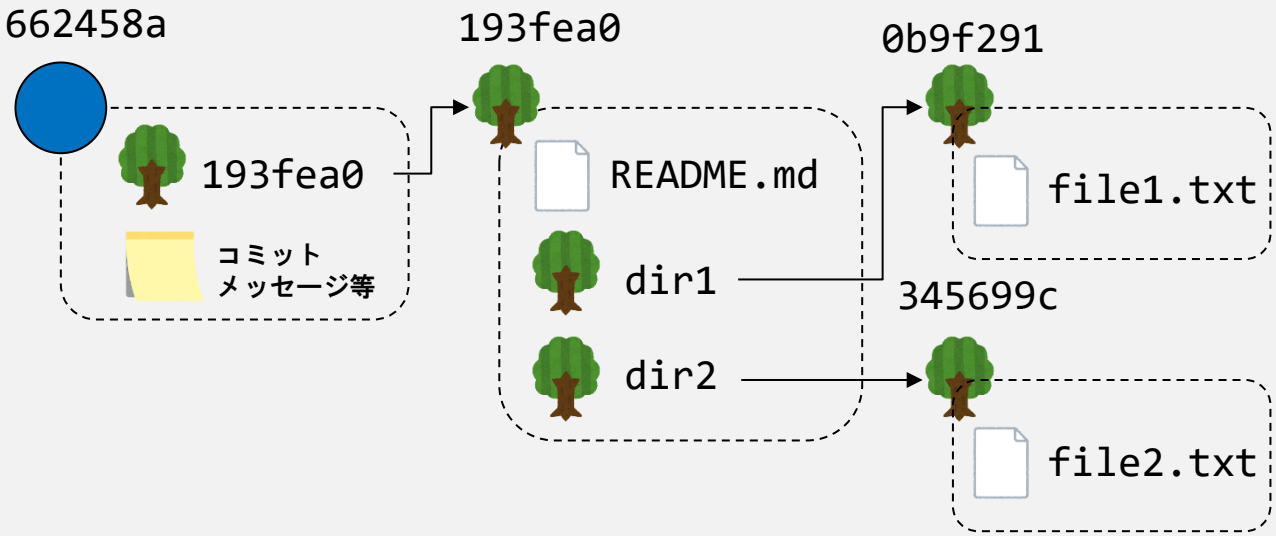


コミットが保存する「スナップショット」を表すのがtreeオブジェクト

treeオブジェクト

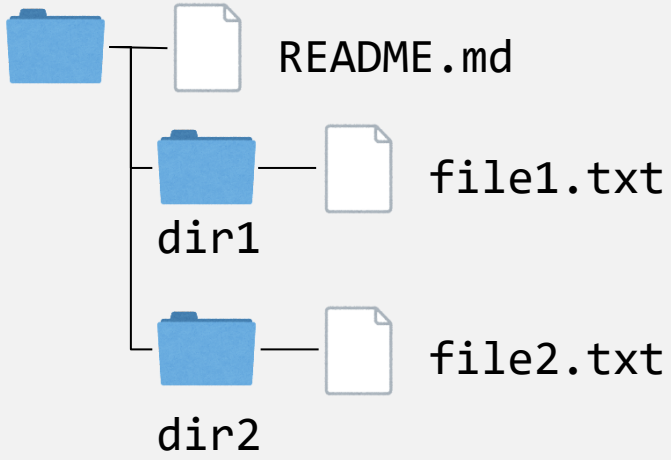
treeオブジェクトの中身を見てみる

```
$ git cat-file -p 193fea0
100644 blob e845566c06f9bf557d35e8292c37cf05d97a9769    README.md
040000 tree 0b9f291245f6c596fd30bee925fe94fe0cbadd60    dir1
040000 tree 345699cffb47ac20257e0ce4cebcbfc4b2a7f9e3    dir2
```

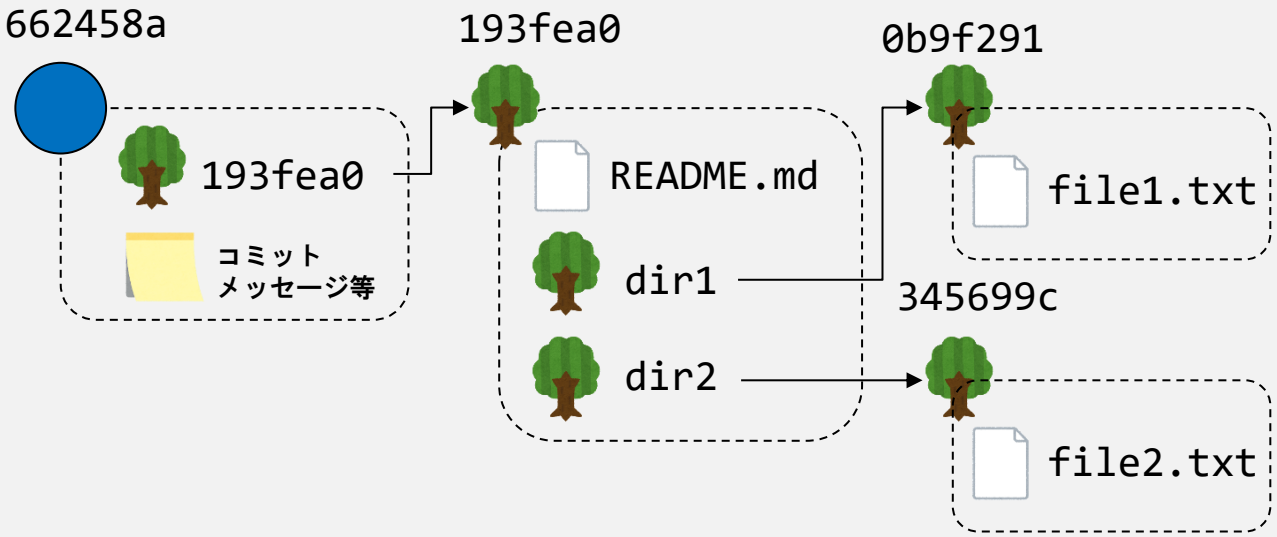


treeオブジェクト

プロジェクトのディレクトリ構造

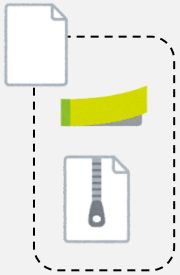


Gitにおけるスナップショット表現



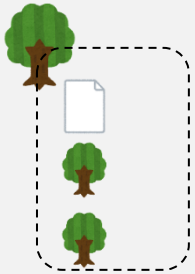
Gitのオブジェクトのまとめ

Gitのオブジェクトは.git/objectsの中に格納されている



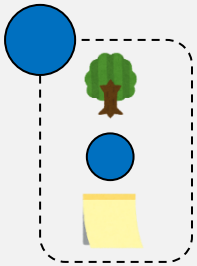
blobオブジェクト

ファイルに対応
ヘッダをつけて圧縮したもの



treeオブジェクト

ディレクトリに対応
スナップショットを表現するのに使う



コミットオブジェクト

スナップショットを表現するtreeオブジェクトと
親コミットの情報、およびコミットメッセージ等を
まとめたもの

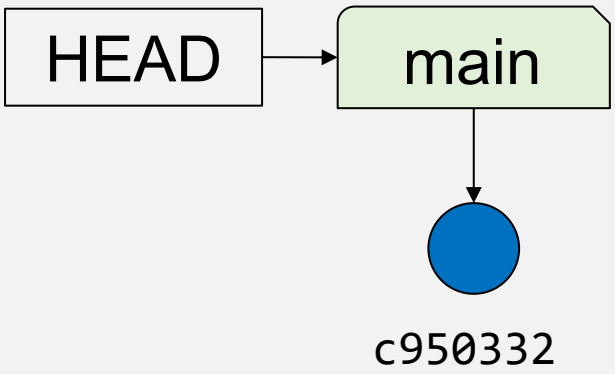
HEADやブランチ

HEADやブランチといったGitの参照はファイルとして保存されている

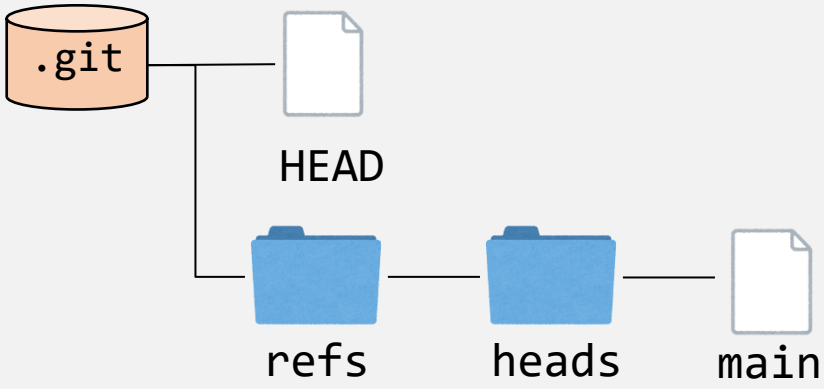
HEAD → `.git/HEAD`

main → `.git/heads/main`

ブランチの状態



ファイルの実体



HEADやブランチ

HEADファイルの中身

```
$ cat .git/HEAD
ref: refs/heads/main
```

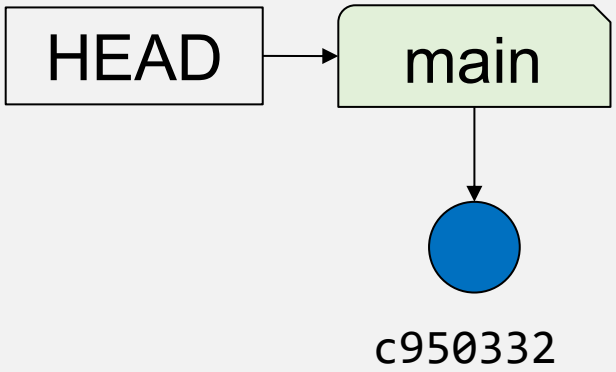
対応するブランチの
実体ファイルへのパス

mainファイルの中身

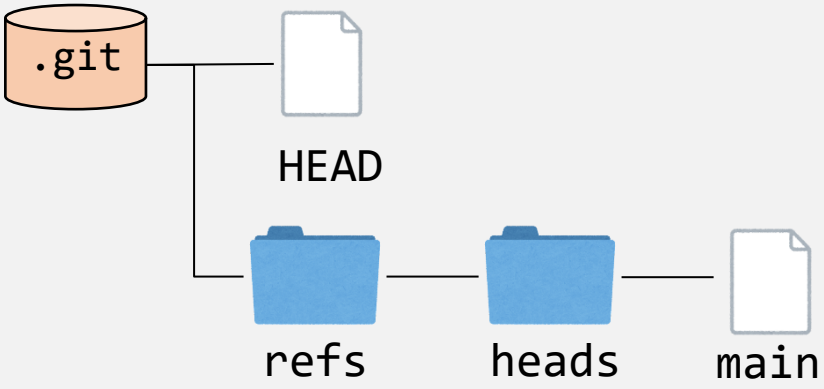
```
$ cat .git/refs/heads/main
c9503326279796b24be86bdf9beb01c1af2d2b95
```

コミットハッシュ

ブランチの状態



ファイルの実体

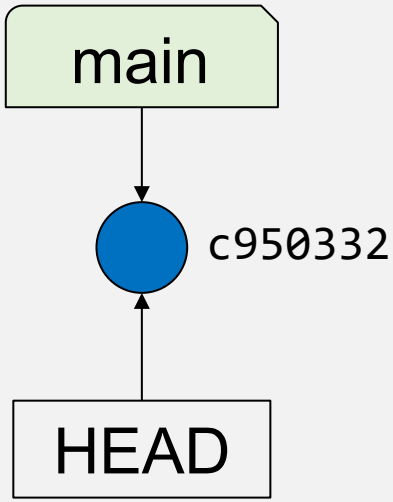


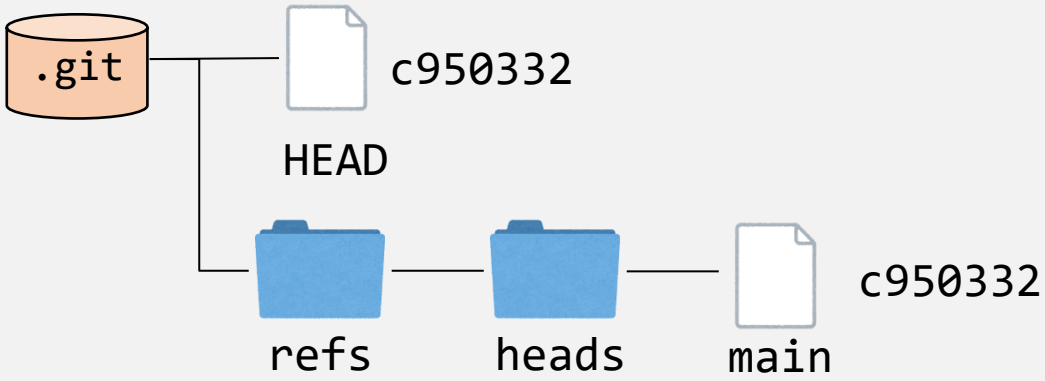
HEADやブランチ

Detached HEAD状態の時

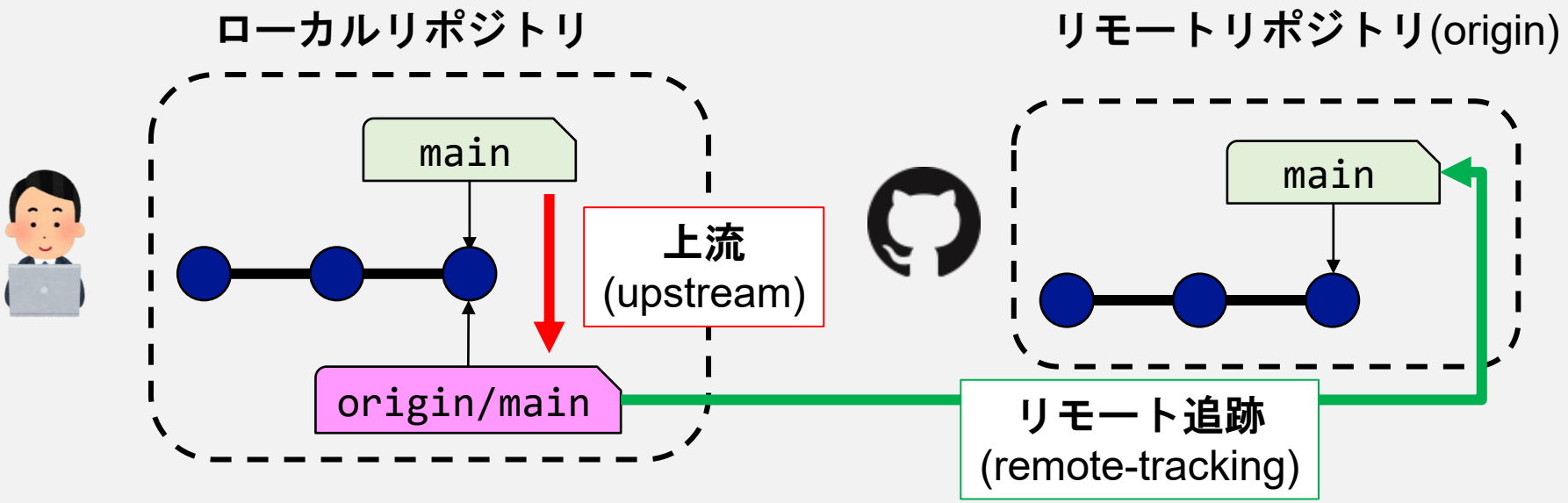
```
$ cat .git/HEAD
c9503326279796b24be86bdf9beb01c1af2d2b95
```

HEADが直接コミットハッシュを指している

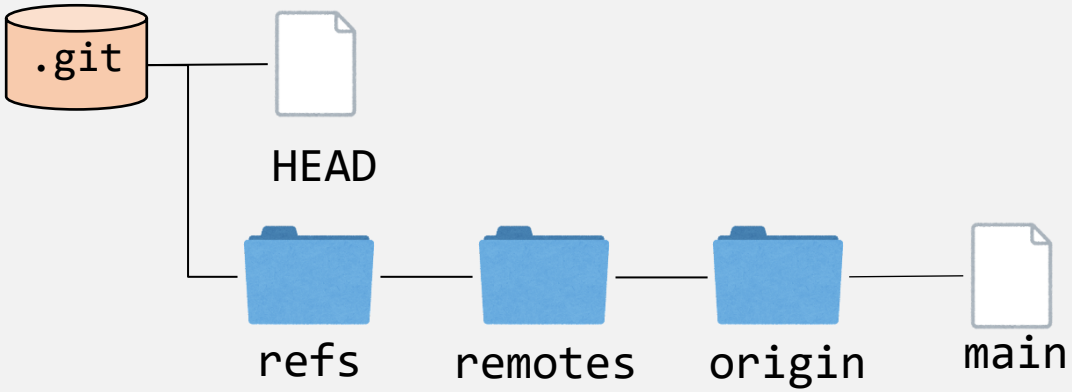




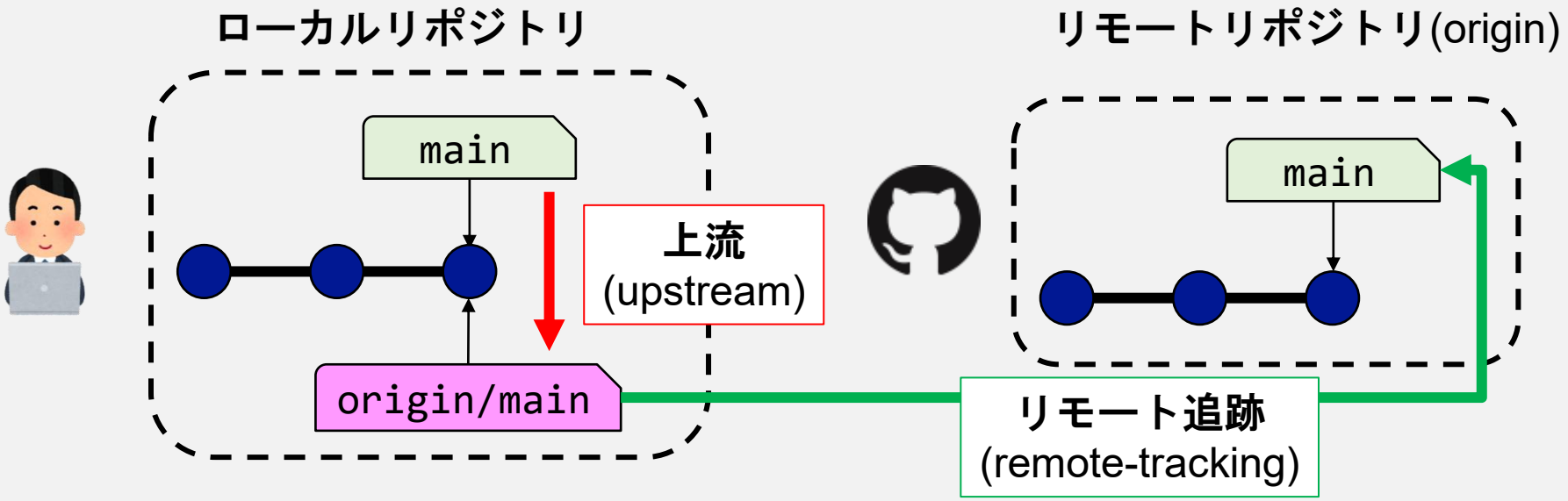
上流ブランチとリモート追跡ブランチ



リモート追跡ブランチの実体は .git/refs/remotesにある

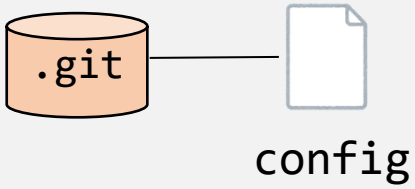


上流ブランチとリモート追跡ブランチ



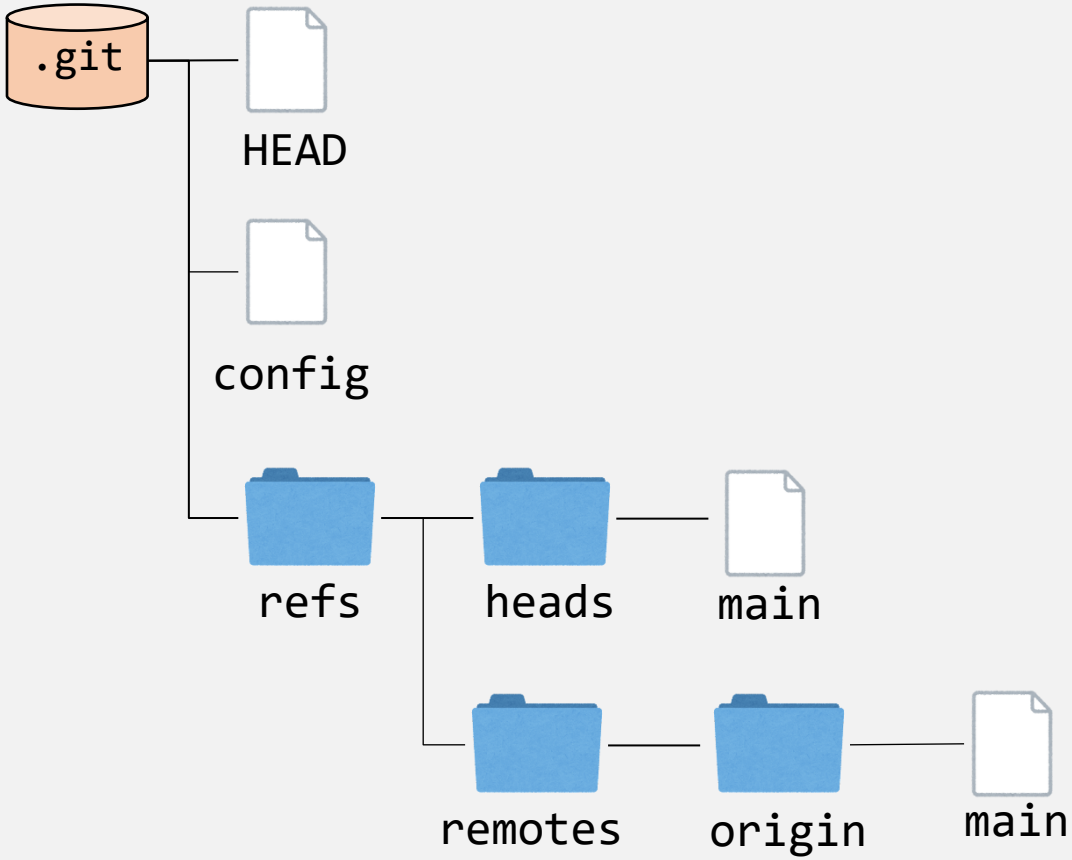
上流ブランチの情報は.git/configに保存されている

```
$ cat .git/config
(snip)
[remote "origin"]
        url = ../test.git
        fetch = +refs/heads/*:refs/remotes/origin/*
[branch "main"]
        remote = origin
        merge = refs/heads/main
```

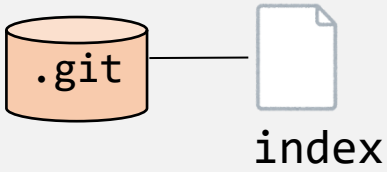


Gitの参照のまとめ

ブランチの実体は.git/refs以下にファイルとして保存されている
HEADの実体は.git/HEADというファイル
上流ブランチの情報は.git/configに保存されている



インデックス

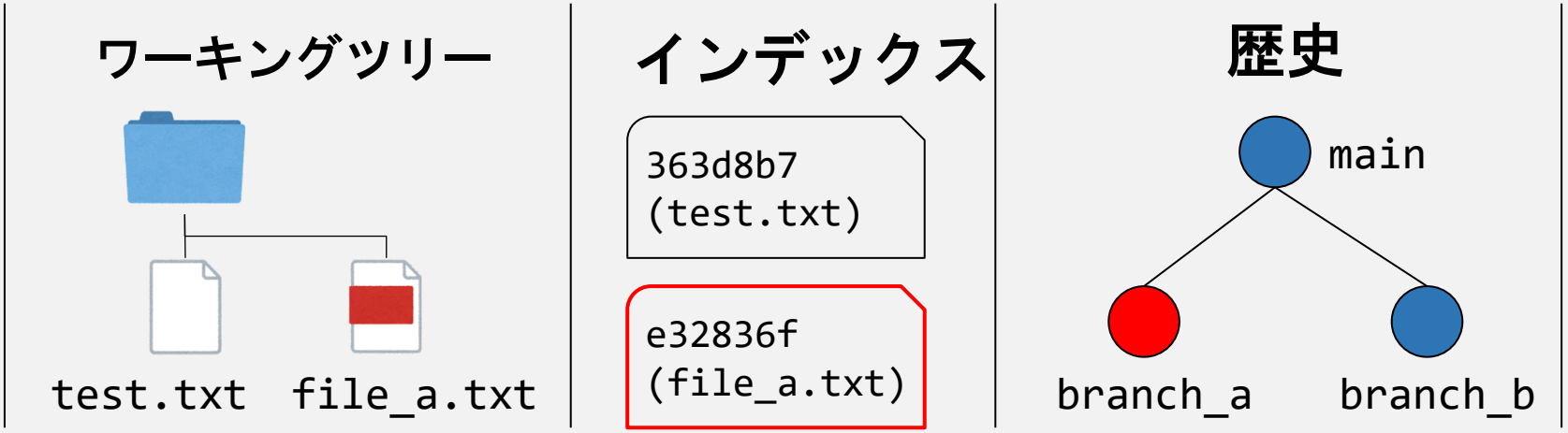


インデックスの実体は.git/index

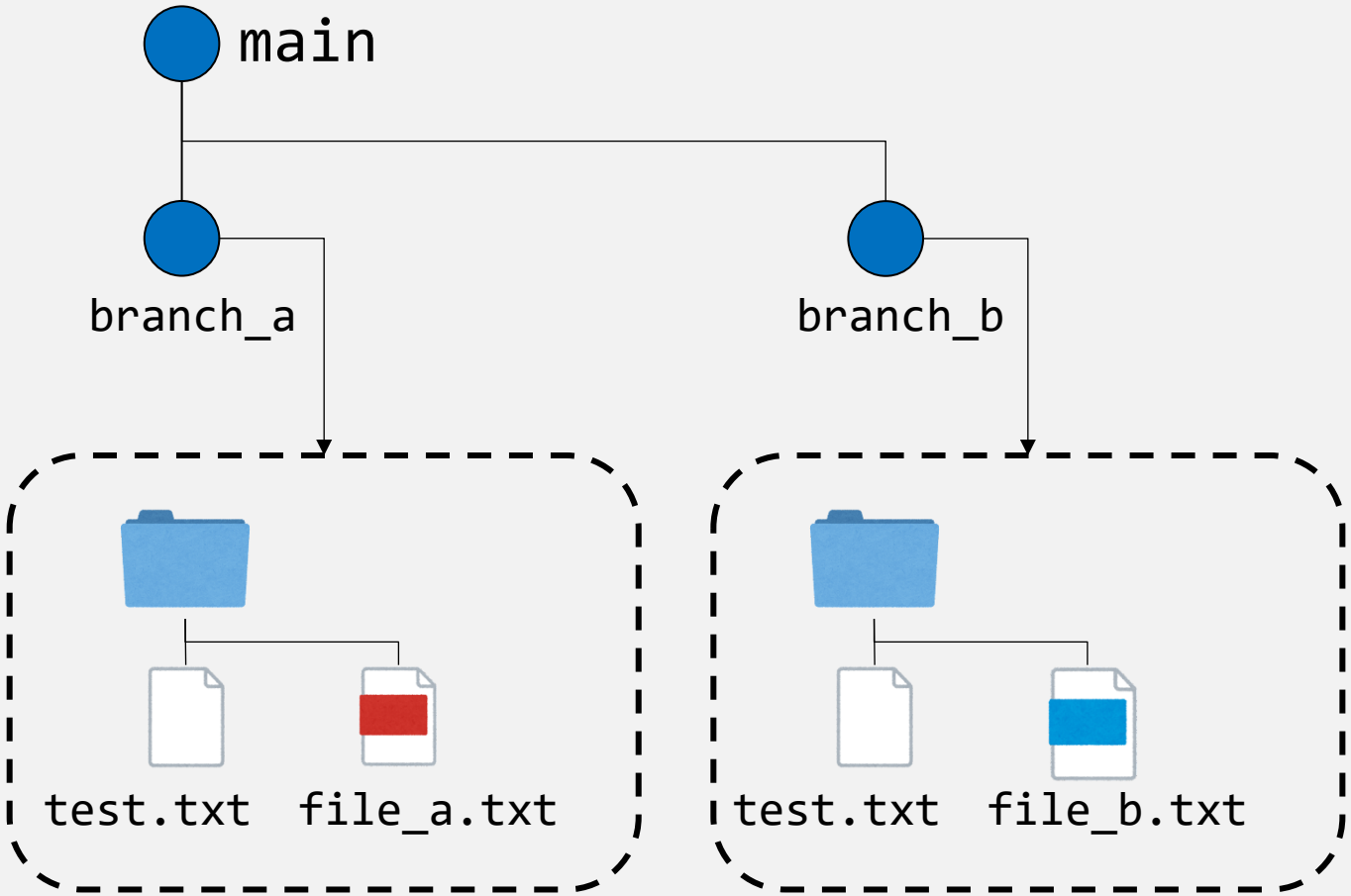
インデックスの中身を表示する

```
$ git ls-files --stage
100644 e32836f4cedd87510bfd2f145bc0696861fdb026 0    file_a.txt
100644 363d8b784900d74b3159e8e93a651c0db42629ef 0    test.txt
```

スナップショットに対応するblobオブジェクトとファイル名が記録されている



インデックス



異なるブランチは、異なるスナップショットを保存している

インデックス

カレントブランチがbranch_aの状態でインデックスの中身を表示

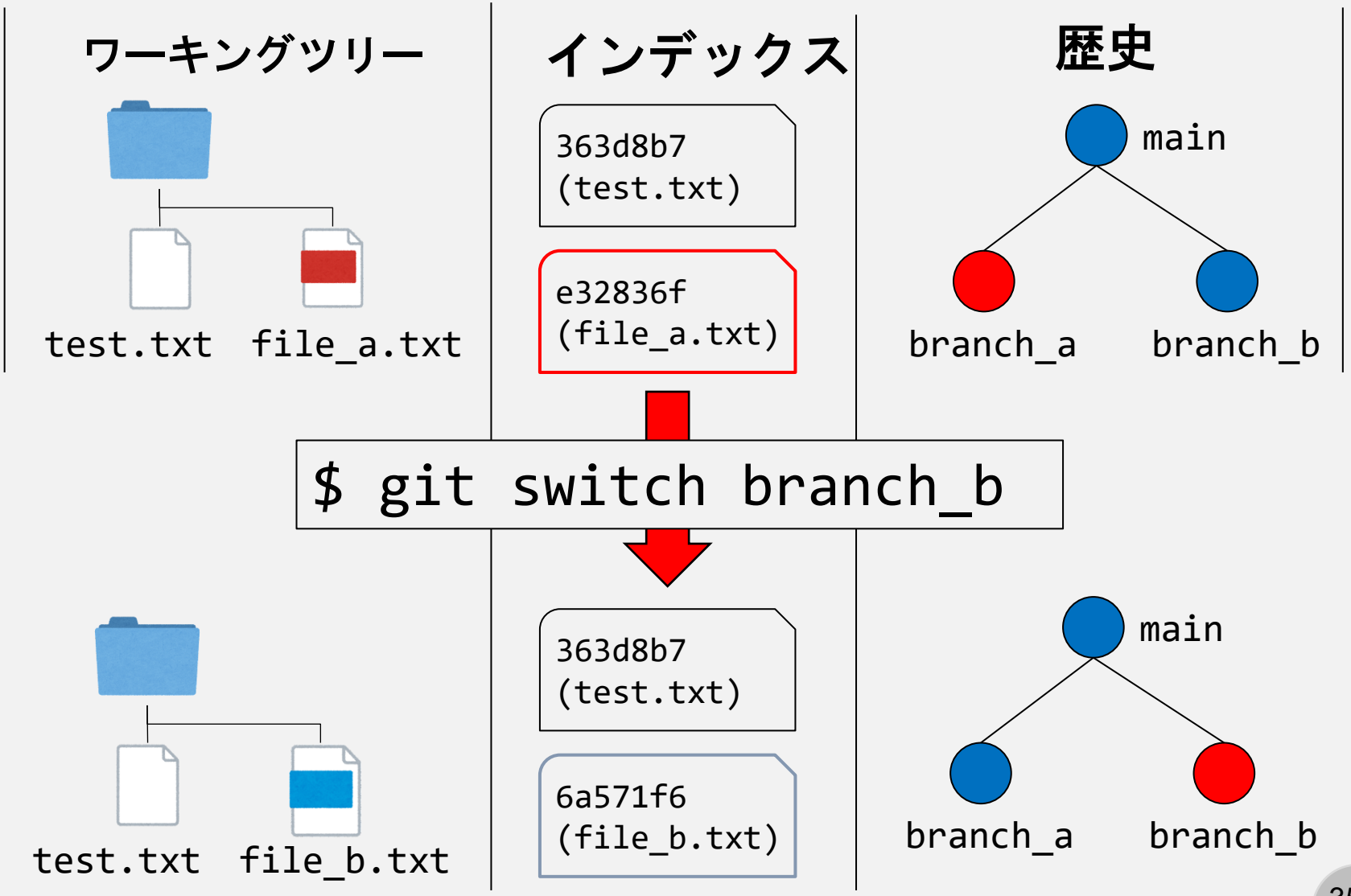
```
$ git ls-files --stage
100644 e32836f4cedd87510bfd2f145bc0696861fdb026 0    file_a.txt
100644 363d8b784900d74b3159e8e93a651c0db42629ef 0    test.txt
```

カレントブランチをbranch_bに切り替えてインデックスの中身を表示

```
$ git switch branch_b
$ git ls-files --stage
100644 6a571f63d9d0bce7995b5c08d218370d7ea719a5 0    file_b.txt
100644 363d8b784900d74b3159e8e93a651c0db42629ef 0    test.txt
```

ブランチを切り替えると、インデックスの中身も切り替わる

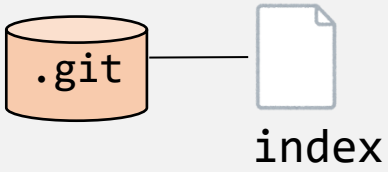
インデックス



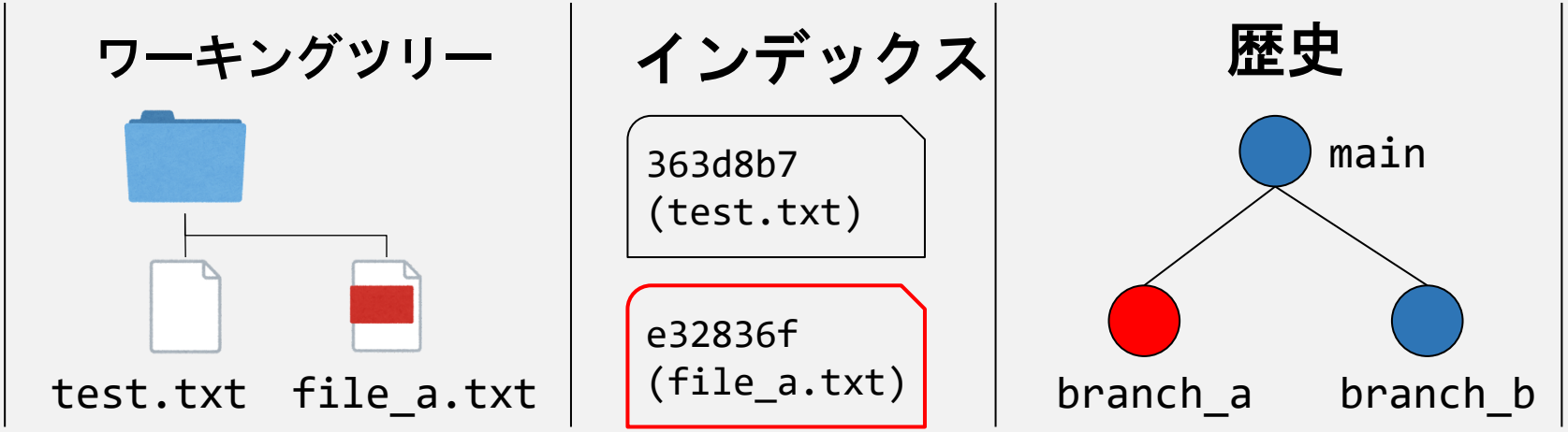
ブランチを切り替えると、インデックスの中身も切り替わる

インデックスのまとめ

インデックスの実体は.git/index
中身はblobオブジェクトとファイル名のリスト
ブランチを切り替えるとインデックスの中身も切り替わる




```
$ git ls-files --stage
100644 e32836f4cedd87510bfd2f145bc0696861fdb026 0    file_a.txt
100644 363d8b784900d74b3159e8e93a651c0db42629ef 0    test.txt
```



まとめ

「.git」の中身を覗いてみた

- Gitのオブジェクト
  - .git/objectsに保存
  - blobオブジェクト：ファイルに対応
  - treeオブジェクト：ディレクトリに対応
  - コミットオブジェクト：必要な情報をまとめたもの
- Gitのブランチ
  - HEADの実体は.git/HEAD
  - ブランチは.git/refsに保存
  - ブランチの実体は同名のファイル
- インデックス
  - 実体は.git/index
  - 中身はblobオブジェクトとファイル名のリスト

Gitの実装は非常に単純かつ素直